

JavaScript 클래스

학습 체크리스트 (1/2)

- ☐ 객체 리터럴 기본 문법, 속성 축약 및 계산된 속성명 활용
- ☐ 객체 및 생성될 인스턴스를 가리키는 `this` 동작 원리 이해
- ☐ 생성자 함수와 `prototype` 객체의 관계 및 프로토타입 상속 구조 파악
- ☐ 메서드 중복 생성 방지를 위한 프로토타입 공유 메서드 패턴 적용
- ☐ ES6 `class` 기본 문법 및 `constructor` 를 통한 인스턴스 초기화

학습 체크리스트 (2/2)



접두사(Private 필드) 기반의 정보 은닉 및 캡슐화 구현



`extends` 와 `super` 를 활용한 클래스 상속 및 메서드 오버라이딩 적용



Getter / Setter 접근자를 통한 데이터 검증 및 안전한 접근 통제



인스턴스 생성 없는 정적 멤버(`static`)의 쓰임새 및 특징 구분



`instanceof` 타입 검사 및 내장 빌트인 클래스 핵심 유틸리티 활용

객체 리터럴

객체 리터럴 (Object Literal)

중괄호 `{}` 를 사용하여 객체를 선언하고, 그 내부에 속성(Property)과 메서드를 정의하는 가장 직관적이고 널리 사용되는 객체 생성 방식

- 동적 추가: 객체가 생성된 후에도 새로운 속성이나 메서드를 자유롭게 추가하고 삭제할 수 있음
- 단축 속성명: 변수명과 객체 속성명이 같을 때 콜론과 중복 명칭 생략 가능
- 계산된 속성명: 대괄호 `[]` 안에 표현식을 넣어 동적으로 속성 이름을 결정할 수 있음

객체 리터럴 구현 예시

```
1  const role = "admin";
2  const nameKey = "userName";
3
4  const user = {
5      role,                // 단축 속성명
6      [nameKey]: "민지",   // 계산된 속성명
7      greet() {           // 메서드 축약 표현
8          return `안녕하세요, ${this.userName}님!`;
9      }
10 };
11
12 console.log(user.greet()); // "안녕하세요, 민지님!"
```

객체 리터럴 특징 및 동작

- 동적 유연성: 런타임에 속성 키를 동적으로 계산하거나 변수 기반 단축 속성 정의 가능
- `this` 바인딩: 객체 내 축약 메서드 내부의 `this` 는 메서드를 호출한 객체를 가리킴

JavaScript의 this

this 키워드 (this)

자신이 속한 객체 또는 자신이 생성할 인스턴스를 가리키는 자기 참조 변수(Self-referencing variable)

- 동적 바인딩: `this`가 가리키는 대상은 함수가 정의된 위치가 아닌 **함수가 호출되는 방식**에 따라 동적으로 결정됨
- 객체 메서드로 호출: 해당 메서드를 실행한 온점(`.`) 앞의 **객체**를 가리킴
- 생성자(클래스)로 호출: `new` 연산자를 통해 **생성될 새 인스턴스 객체** 자체를 가리킴

객체 메서드에서의 this

```
1  const user = {  
2    name: "지수",  
3    greet() {  
4      console.log(this.name); // 호출한 객체(user) 참조  
5    }  
6  };  
7  
8  user.greet(); // "지수"
```


생성자/클래스에서의 this

```
1  class Person {  
2    constructor(name) {  
3      this.name = name; // 생성될 새 인스턴스 참조  
4    }  
5  }  
6  
7  const p = new Person("민수");  
8  console.log(p.name); // "민수"
```

프로토타입 기반 상속

프로토타입 (Prototype)

모든 객체가 상위 객체 역할을 하는 숨겨진 프로토타입 객체(`[[Prototype]]`)를 통해 속성과 메서드를 상속받아 공유하는 구조

- **프로토타입 기반**: JavaScript는 클래스 기반 상속 대신 프로토타입 링크를 통해 객체의 기능을 확장함
- **상속 체인**: 객체에 특정 속성이나 메서드가 없으면 상위 프로토타입을 따라 올라가며 탐색함
- **메모리 절약**: 인스턴스마다 메서드를 개별 생성하지 않고 프로토타입에 선언하여 메모리를 공유할 수 있음

생성자 함수와 메모리 낭비

- 생성자 함수: 동일한 형태의 객체를 찍어내기 위해 정의하는 템플릿용 함수
- 중복 생성 문제: 생성자 내부에서 메서드를 선언하면, 인스턴스를 생성할 때마다 동일한 기능을 하는 독립적인 함수가 매번 새로 메모리에 할당되어 메모리 낭비 발생

생성자 함수 구현 예시

```
1  function Person(name) {  
2    this.name = name;  
3    this.greet = function() { // 인스턴스 생성 시마다 중복 할당  
4      return `안녕, 난 ${this.name}야.`;  
5    };  
6  }  
7  
8  const userA = new Person("지민");  
9  const userB = new Person("태형");  
10  
11 console.log(userA.greet === userB.greet); // false (서로 다른 메모리 참조)
```

프로토타입을 통한 공유 메서드

- 공유 메서드 선언: 생성자 함수의 `prototype` 객체에 메서드를 등록하여 모든 자식 인스턴스가 상속받게 함
- 메모리 효율화: 단 하나의 메서드 메모리를 모든 자식이 공유하므로, 불필요한 함수 재생성을 원천 방지하여 성능 극대화

공유 메서드 구현 예시

```
1  function Person(name) {
2    this.name = name;
3  }
4
5  // 생성자 프로토타입에 공용 메서드 추가
6  Person.prototype.greet = function() {
7    return `안녕, 난 ${this.name}야.`;
8  };
9
10 const userA = new Person("정국");
11 const userB = new Person("남준");
12
13 console.log(userA.greet === userB.greet); // true (동일한 공유 메서드 참조)
```

클래스 문법의 도입

클래스 (Class)

기존의 프로토타입 기반 상속 구조를 다른 OOP 언어들처럼 친숙하고 명확한 문법으로 구현할 수 있도록 포장한 문법적 설탕(Syntactic Sugar)

- **직관적인 OOP 구현:** 프로토타입 링크를 직접 다루는 대신 정형화된 클래스 키워드로 객체 템플릿 정의 가능
- **가독성 향상:** 상속 관계 및 생성자, 메서드, 속성이 명확하게 구분되어 협업에 유리함
- **추가적 보안:** `class` 내에서는 캡슐화와 정보 은닉을 돕는 여러 안전장치가 제공됨

클래스 생성자 (Constructor)

- `constructor` : 클래스로 인스턴스를 생성할 때 가장 먼저 1회 자동 실행되어 객체를 초기화하는 특수 메서드
- 기본 문법: 클래스 내부에 오직 하나만 존재할 수 있으며, 생략 시 자동으로 빈 생성자가 삽입됨

```
1  class Car {  
2    // 생성자 메서드 정의  
3    constructor(model, color) {  
4      this.model = model;  
5      this.color = color;  
6    }  
7  }  
8  
9  const myCar = new Car("아이오닉", "화이트");  
10 console.log(myCar.model); // "아이오닉"
```


클래스 필드 (Public vs Private)

- 필드 (Field): 클래스 내부에 선언되는 속성. `constructor` 외부에서도 직접 정의 가능
- Public 필드: 기본 속성으로 외부에서 제약 없이 읽고 쓸 수 있음
- Private 필드 (`#`): 변수명 앞에 `#`을 붙여 정의하며, 클래스 내부에서만 접근 가능. 외부에서 접근 시 에러 발생
- 캡슐화의 완성: 정보 은닉을 실현하여 객체 내부 데이터를 안전하게 보호함

클래스 필드 구현 예시

```
1  class BankAccount {
2      owner;          // Public 필드
3      #balance = 0; // Private 필드 (최상단 선언 필수)
4
5      constructor(owner, initialDeposit) {
6          this.owner = owner;
7          this.#balance = initialDeposit;
8      }
9
10     showBalance() {
11         return this.#balance; // 클래스 내부 접근 가능
12     }
13 }
```

클래스 필드 활용 예시

```
1  const account = new BankAccount("수지", 50000);  
2  
3  console.log(account.owner); // "수지" (접근 가능)  
4  
5  // console.log(account.#balance); // 문법 에러 (외부 접근 제한)  
6  
7  console.log(account.showBalance()); // 50000 (우회 확인)
```

클래스 상속과 오버라이딩

- 상속 (`extends`): 기존 클래스(부모)의 속성과 메서드를 그대로 물려받아 파생 클래스(자식)를 생성하는 기법
- `super()`: 자식 클래스의 `constructor` 내부에서 부모 생성자를 먼저 실행시키기 위해 반드시 호출해야 하는 함수
- 메서드 오버라이딩: 부모로부터 상속받은 메서드를 자식 클래스의 특성에 맞게 새로 정의(덮어쓰기)하는 행위

클래스 상속 구현 예시

```
1  class Animal {
2      constructor(name) { this.name = name; }
3      speak() { return `${this.name}가 소리를 냅니다.`; }
4  }
5
6  class Dog extends Animal {
7      constructor(name, breed) {
8          super(name); // 부모 생성자 실행 (this 사용 전 필수)
9          this.breed = breed;
10     }
11     speak() { // 메서드 오버라이딩 (재정의)
12         return `${this.name}(${this.breed})가 멍멍 짭니다!`;
13     }
14 }
```

클래스 상속 활용 예시

```
1  const myDog = new Dog("바둑이", "진돗개");  
2  
3  console.log(myDog.speak());  
4  // 출력: "바둑이(진돗개)가 멍멍 짭니다!"
```

접근자 프로퍼티 (Getter & Setter)

- **Getter** (`get`): 속성 값을 읽으려고 할 때 자동으로 실행되는 메서드. 가공된 값을 외부로 안전하게 반환함
- **Setter** (`set`): 속성 값을 대입하려고 할 때 실행되는 메서드. 전달된 값 검증 및 방어 코드 수립 가능
- **변수명 충돌 방지**: Getter/Setter 내부의 무한 루프를 피하기 위해 통상 실제 저장 변수명에 `_` 접두사를 붙임

Getter & Setter 구현 예시

```
1  class Person {
2      constructor(name, age) {
3          this.name = name;
4          this.age = age; // setter 호출
5      }
6
7      get age() { return this._age; }
8
9      set age(value) {
10         // 음수가 입력되면 0으로 보정하는 안전장치
11         this._age = value < 0 ? 0 : value;
12     }
13 }
```


Getter & Setter 활용 예시

```
1  const user = new Person("하니", -5);  
2  
3  console.log(user.age); // 0 (음수 방어 작동)
```

정적 속성 및 메서드 (Static)

- 정적 멤버 (`static`): 인스턴스에 속하지 않고 클래스 자체에 직접 연결되는 멤버
- 특징: `new` 를 통한 인스턴스 생성 없이 `클래스명.속성` 또는 `클래스명.메서드()` 형태로 즉시 사용 가능
- 용도: 여러 인스턴스가 공통으로 참조할 상수나 유틸리티형 함수 설계 시 메모리 효율 극대화

```
1 class Calculator {  
2     static PI = 3.141592;  
3     static add(a, b) { return a + b; }  
4 }  
5  
6 console.log(Calculator.PI); // 3.141592  
7 console.log(Calculator.add(10, 20)); // 30
```

클래스 기반 타입 확인

- `instanceof` : 객체가 특정 클래스 또는 부모 클래스로부터 파생된 인스턴스인지 여부를 식별하여 불리언으로 반환
- `constructor` : 인스턴스가 자신을 생성한 클래스/생성자 함수를 직접 바라보게 하는 속성

```
1 class Vehicle {}  
2 class Airplane extends Vehicle {}  
3 const jet = new Airplane();  
4  
5 console.log(jet instanceof Airplane); // true  
6 console.log(jet instanceof Vehicle); // true (부모 타입 만족)  
7 console.log(jet.constructor === Airplane); // true
```

내장 클래스 타입 검사

- 타입 판단 한계 보완: `typeof` 가 배열을 단순 `"object"` 로 반환하는 문제를 방지하는 내장 클래스 전용 정적 메서드
- `Array.isArray()` : 배열 여부를 가장 확실하게 검증
- `Number.isInteger()` / `Number.isNaN()` : 전달된 값이 정수인지, 또는 실제 `NaN` 데이터인지 판별

```
1  const arr = [1, 2, 3];
2  console.log(typeof arr);           // "object" (타입 식별 불가)
3  console.log(Array.isArray(arr));   // true (배열 판별 정확)
4
5  console.log(Number.isInteger(10.5)); // false
6  console.log(Number.isNaN(NaN));     // true
```

명시적 형변환 및 문자열 파싱

- 명시적 형변환: `new` 없이 `Number()`, `String()`, `Boolean()` 함수를 호출하여 데이터 타입을 즉시 변환
- 문자열 파싱: `Number.parseInt()`, `Number.parseFloat()` 로 문자열의 시작점부터 유효한 숫자를 정밀 추출

```
1  const num = Number("123");    // "number"
2  const isTrue = Boolean("A");  // true (Truthy)
3  const str = String(500);      // "string"
4
5  console.log(Number.parseInt("42px"));    // 42
6  console.log(Number.parseFloat("3.14원")); // 3.14
```

인스턴스 복제와 스프레드 주의사항

- **프로토타입 유실**: 인스턴스를 스프레드 연산자(`...`)로 얹은 복사하면 프로토타입 공유 메서드와 Getter/Setter가 끊긴 순수 객체(Plain Object)가 되어버림
- **올바른 복제**: 클래스 내부에 인스턴스를 직접 재생성하여 반환하는 수동 복제 메서드(`clone()`)를 별도로 정의하여 사용해야 함

인스턴스 복제 구현 예시

```
1  class Warrior {
2    constructor(name) { this.name = name; }
3    attack() { return `${this.name} 공격!`; }
4    clone() { return new Warrior(this.name); }
5  }
6
7  const p1 = new Warrior("전사");
8  const badClone = { ...p1 }; // 얕은 복사
9  // badClone.attack();      // 에러 (메서드 유실)
10
11 const goodClone = p1.clone();
12 console.log(goodClone.attack()); // "전사 공격!" (정상 작동)
```

빌트인 클래스(Object) 유틸리티

- `Object.keys(obj)` : 객체의 모든 속성 이름(키)을 담은 배열 반환
- `Object.values(obj)` : 객체의 모든 속성 값을 담은 배열 반환
- `Object.entries(obj)` : [키, 값] 쌍의 2차원 배열 반환
- `Object.freeze(obj)` : 객체를 완전히 고결(동결)하여 신규 추가, 삭제, 기존 값 수정을 사전에 완벽 차단

```
1  const student = { name: "민지", grade: 3 };
2  console.log(Object.keys(student)); // ["name", "grade"]
3  console.log(Object.values(student)); // ["민지", 3]
4
5  const config = { api: "https://api.com" };
6  Object.freeze(config);
7  config.api = "http://hack.com"; // 수정 무시됨
8  console.log(config.api); // "https://api.com"
```


빌트인 클래스(Array) 유틸리티

- `Array.from(likeArray)` : 유사 배열 객체(arguments, NodeList 등)나 반복 가능한 객체를 진짜 배열 객체로 복사/변환하여 강력한 배열 고차함수 사용 권한을 획득하게 함

```
1  function sumAll() {  
2    // 유사 배열 arguments를 진짜 배열로 변환  
3    const argsArray = Array.from(arguments);  
4    return argsArray.reduce((acc, cur) => acc + cur, 0);  
5  }  
6  
7  console.log(sumAll(10, 20, 30)); // 60
```